# Comparing Plain CNN and ResNet Models at Different Depths for CIFAR-10 Image Classification

Zahraa Ezzat
*2201131107*

Seba Alzaher
*2201114862*

## I. Introduction

Image classification is one of the most important tasks in computer vision because it is used in many applications such as object recognition, monitoring systems, and image-based automation. Convolutional Neural Networks (CNNs) are widely used for this task because they can learn useful image features directly from raw images.

However, as CNNs became deeper, they often achieved better performance, but after a certain point, plain networks start getting worse because they become too hard to train and sometimes give worse results than shallower networks. This issue is called the degradation problem.

To deal with this issue, residual networks (ResNet) introduced shortcut connections that let information move more easily through the network. These shortcut connections make the learning process smoother, make deeper models easier to optimize, and help improve image classification performance.

In this project, we compare plain CNN models and ResNet models on the CIFAR-10 dataset. At first, we compare Plain CNN-18 with ResNet-18. Then, to better test the effect of depth, we add a deeper comparison using Plain CNN-56 and ResNet-56.

Unlike the original ResNet paper, which mainly reported error rates, this project also uses accuracy, precision, recall, F1-score, and confusion matrices to give a clearer view of the models' class-level performance and mistakes.

## II. Related Work

### A. Early architectures

Early convolutional neural networks showed that deep networks could perform very well, but they were still quite simple in how they stacked layers and that they did not directly solve the problem of training much deeper networks [4].

Later, models like GoogLeNet tried to make networks better by using more complex designs called Inception to improve accuracy while keeping computation more efficient [5].

### B. ResNet paper

The ResNet paper is the most important part for our project. The authors showed that deeper plain networks can become harder to train when adding too many layers and may stop learning or even perform worse. In their experiments, a deeper plain network had higher training error than a shallower plain network, which means the problem was not only overfitting.

They solved this by adding skip, or residual, connections which are shortcut paths that help information to move more easily through the network allowing the network to keep the original information while learning new details [1].

The paper showed that residual networks performed better than plain networks, especially when the depth increased.

### C. CIFAR datasets

Some later studies compared ResNet-based models with other CNN architectures on image datasets such as CIFAR-10 and CIFAR-100. One study on CIFAR-10 compared an AlexNet-based model with ResNet-based model and the study eventually reported that a ResNet-based model achieved higher test accuracy [2]. Another study on CIFAR-100 compared ResNet-18, GoogLeNet, and VGG-16 and found that ResNet-18 is the best overall model [3]. These studies support the idea that residual connections can improve learning and classification performance.

### D. Later architectures influenced by ResNet

Later architectures continued the general idea of improving information flow between layers such as DenseNet which connected each layer to all previous layers instead of using only additive skip connections showing that the idea introduced by ResNet had an important influence on later CNN architectures [6].

## III. Novelty Statement

The original ResNet paper showed that residual connections help deep neural networks train better and improve image classification results [1].

In our project, our contribution is an experimental evaluation of how residual connections affect CNN performance at different depths on CIFAR-10. Instead of only comparing final accuracy, we compare Plain CNN and ResNet models using two depths, 18 and 56 layers, and evaluate them using accuracy, precision, recall, F1-score, confusion matrices, training loss curves, and test accuracy curves.

The 18-layer comparison gives a basic view of how ResNet performs against a plain CNN when the model is not extremely deep. The 56-layer comparison is added to better show the effect of residual connections when depth increases.

## IV. Methodology

### A. How ResNet Works

In a plain CNN, each layer tries to learn the direct mapping from the input $x$ to the output $H(x)$, so the output can be written as $y = H(x)$.
This means the stacked layers transform the input into a new representation. So when the network becomes deeper, learning this direct mapping can become harder because the information has to pass through many stacked layers without any shortcut path.
This can make optimization more difficult, since the model must learn how to transform the input while also keeping useful information from earlier layers.
In ResNet, the layers only have to learn the "residual" (the difference) between the input and the output, i.e. the stacked layers learn the residual function instead of the full direct mapping.
The residual function is written as: $F(x) = H(x) - x$. So the final output becomes: $y = F(x) + x$, where $x$ is the shortcut that carries the original information forward. This shortcut prevents the network from forgetting what it already knows as it gets deeper.

### B. Proposed Extension - Our Comparison Plan

For our extension, we compare plain CNN models and ResNet models on CIFAR-10. In the first experiment, we compare Plain CNN-18 with ResNet-18. This was our original comparison, where both models have similar depth, but ResNet-18 uses shortcut connections while Plain CNN-18 does not.
After reviewing the 18-layer results and the original ResNet paper more carefully, we noticed that the effect of residual connections is expected to become clearer when the network depth increases. Because of this, we added a second experiment using Plain CNN-56 and ResNet-56. The goal was to test whether increasing the depth makes the plain CNN harder to train, and whether shortcut connections help ResNet handle the deeper architecture better.
So the final comparison includes four models:

- Plain CNN-18
- ResNet-18
- Plain CNN-56
- ResNet-56

We evaluate these models using accuracy, precision, recall, F1-score, confusion matrices, training loss curves, and test accuracy curves. This allows us to compare not only the final performance, but also how the models learn during training and which classes they confuse the most.

### C. Dataset

We are using CIFAR-10. It has 60,000 color images divided into 10 different classes which are: plane, car, bird, cat, deer, dog, frog, horse, ship, and truck. We split these into 50,000 training images and 10,000 testing images.

Before training, the CIFAR-10 images were converted into PyTorch tensors using TorchVision's `ToTensor()` transform. This makes the images usable by the PyTorch models and changes the pixel values from the normal 0–255 range into values between 0 and 1. After that, the images were normalized using a mean of 0.5 and a standard deviation of 0.5 for each RGB channel. This makes the pixel input values more balanced and helps the models train more smoothly.
During training, we also used data augmentation such as random cropping and random horizontal flipping. These augmentation techniques create slightly different versions of the training images, which helps improve the generalization ability of the models.

### D. Evaluation Metrics

To compare the models, we used more than one evaluation metric. Accuracy alone is useful, but it does not always show the full picture, so we also used precision, recall, F1-score, confusion matrices, training loss curves, and test accuracy curves.

- Accuracy: shows how many images the model classified correctly out of all test images.
- Precision: shows how many of the model's positive predictions were actually correct. It tells how careful the model is when it predicts a class.
- Recall: shows how many images from a class the model was able to correctly find. It tells how many correct examples the model did not miss.
- F1-score: combines precision and recall into one score. It is useful when we want a balanced measure between them.
- Confusion matrix: shows the correct and wrong predictions for each class. This helps to see which classes the model confuses with each other.
- Training loss curve: shows how the model's error changes during training. If the loss goes down, it usually means the model is learning.
- Test accuracy curve: shows how the model performs on the test set after each epoch. This helps to see whether the model is improving over time.

## V. Implementation Environment

The implementation was done in Google Colab using the Python programming language. We mainly used PyTorch, TorchVision, scikit-learn, Matplotlib, and Pandas.
TorchVision was used in two main ways. First, it was used to load the CIFAR-10 dataset and apply the image preprocessing steps before training. Second, for the 18-layer experiment, TorchVision was also used to get the standard ResNet-18 model instead of manually writing the whole model from the referenced GitHub repository since that repository was more educational, while TorchVision gave us a clearer and easier starting point for the actual experiment.
Since CIFAR-10 images are small (32x32), we adjusted the

default ResNet-18 by changing the first convolution layer, removing the early max-pooling layer, and changing the final output layer so that the model predicts 10 classes.

For the plain CNN model, we built our own Plain CNN-18 and Plain CNN-56 models in PyTorch using standard layers such as convolution layers, batch normalization, ReLU, and a final fully connected layer.

For the 56-layer experiment, ResNet-56 was also built manually in PyTorch because TorchVision does not directly provide the CIFAR-style ResNet-56 architecture.

The main difference between Plain CNN-56 and ResNet-56 is that ResNet-56 uses shortcut connections, while Plain CNN-56 does not. For evaluation and visualization, we used scikit-learn to calculate accuracy, precision, recall, F1-score, and confusion matrices. We also used Matplotlib to plot the training loss and test accuracy curves.

## VI. Experiment and Final Results

In Phase 3, we only reported preliminary results using 5 epochs. In Phase 4, we replaced those preliminary results with final experiments using 20 epochs. The final results are shown in Table I.

For the 18-layer experiment, ResNet-18 performed better than Plain CNN-18. ResNet-18 reached 90.96% accuracy, while Plain CNN-18 reached 86.10%.

This means ResNet-18 improved the accuracy by about 4.86 percentage points. The difference is clear, but it is not extremely large. This makes sense because 18 layers is not very deep, so the plain CNN can still train fairly well.

The 56-layer experiment showed a much larger difference. Plain CNN-56 reached only 63.01% accuracy, while ResNet-56 reached 83.80% accuracy.

This means ResNet-56 improved the accuracy by about 20.79 percentage points compared to Plain CNN-56. This result is more important because it shows that when the network becomes deeper, the plain CNN struggles much more, while ResNet can still train effectively.

Figure 1 shows the training loss comparison for all four models. ResNet-18 had the lowest training loss overall. ResNet-56 also trained much better than Plain CNN-56. The Plain CNN-56 loss stayed much higher, which shows that the deeper plain model had difficulty learning.

Figure 2 shows the test accuracy comparison for all four models. ResNet-18 achieved the highest final accuracy, but the main point is the difference between the 56-layer models. Plain CNN-56 stayed much lower, while ResNet-56 improved more smoothly and reached a much stronger result.

Figures 3 and 4 show the confusion matrices for the 56-layer models. Plain CNN-56 made many mistakes between visually similar classes such as cat, dog, bird, deer, and horse. ResNet-56 reduced many of these mistakes and performed better overall.

The original paper showed that deeper plain networks can suffer from the degradation problem, where adding more layers leads to higher training and test error. It also showed that residual networks can handle increased depth better. Our results follow the same general idea. The 18-layer models were relatively close, but when the depth increased to 56 layers, Plain CNN-56 performed much worse while ResNet-56 stayed much stronger. This supports the original paper's main finding.

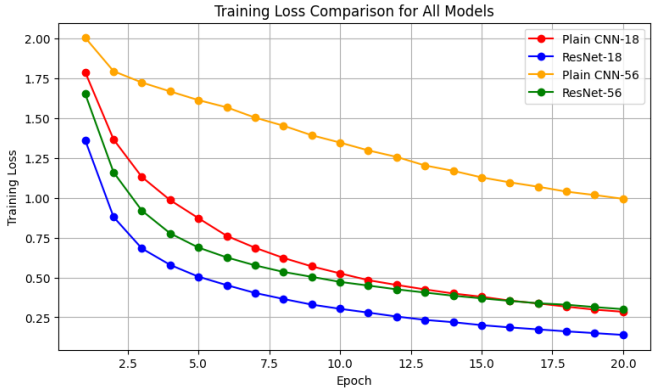


Fig. 1. Training loss comparison for all models over 20 epochs.

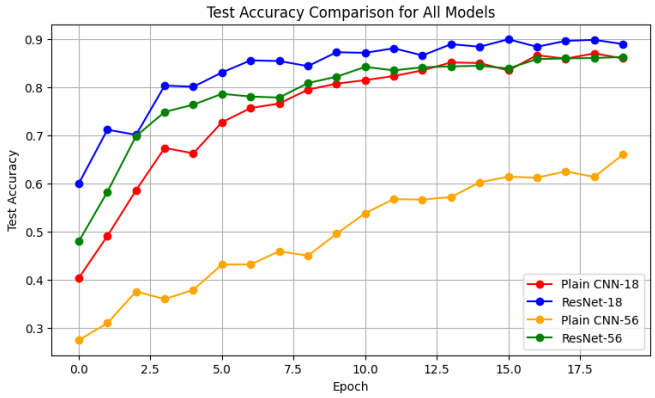


Fig. 2. Test accuracy comparison for Plain CNN-18, ResNet-18, Plain CNN-56, and ResNet-56 over 20 epochs.

## VII. Discussion

The results show that residual connections helped improve the performance, especially when the network became deeper.

In the 18-layer experiment, ResNet-18 performed slightly better than Plain CNN-18. However, the difference was not very large because 18 layers is not deep enough to clearly show the degradation problem. This matches the idea from the original ResNet paper, where 18-layer plain and residual networks were close in performance, while deeper models showed a clearer difference.

The 56-layer experiment was more important for our project. Plain CNN-56 performed much worse than Plain CNN-18, even though it was deeper. This shows that simply adding more layers does not always improve performance. Instead, the deeper plain network became harder to optimize. On the other hand, ResNet-56 still achieved strong performance because the shortcut connections helped information flow through the network and made training easier.

TABLE I
FINAL RESULTS AFTER 20 EPOCHS ON CIFAR-10

| Model | Epochs | Accuracy | Precision | Recall | F1-score |
|---|---|---|---|---|---|
| Plain CNN-18 | 20 | 0.8610 (86.10%) | 0.8631 (86.31%) | 0.8610 (86.10%) | 0.8590 (85.90%) |
| ResNet-18 | 20 | 0.9096 (90.96%) | 0.9114 (91.14%) | 0.9096 (90.96%) | 0.9097 (90.97%) |
| Plain CNN-56 | 20 | 0.6301 (63.01%) | 0.6408 (64.08%) | 0.6301 (63.01%) | 0.6232 (62.32%) |
| ResNet-56 | 20 | 0.8380 (83.80%) | 0.8444 (84.44%) | 0.8380 (83.80%) | 0.8322 (83.22%) |

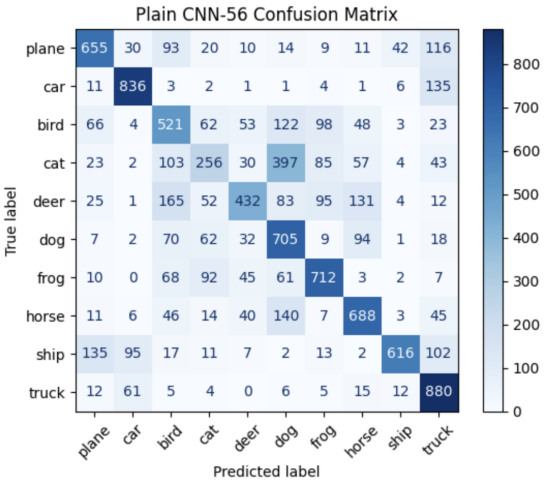


Fig. 3. Confusion matrix for Plain CNN-56.

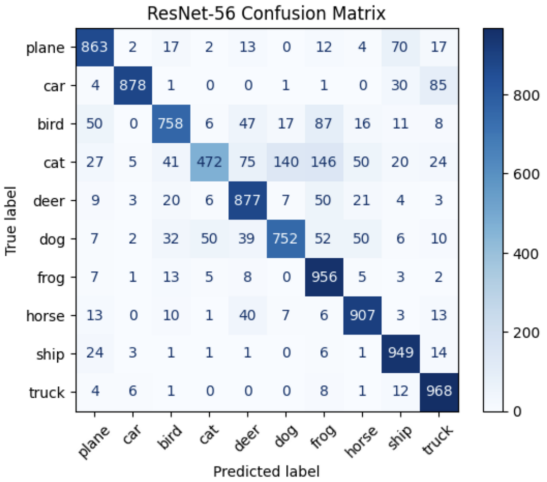


Fig. 4. Confusion matrix for ResNet-56.

One important point is that ResNet-18 had the highest accuracy in our experiment, not ResNet-56. This does not mean that the 56-layer experiment failed. The purpose of the 56-layer experiment was not to beat ResNet-18, but to test whether residual connections help when depth increases. The result clearly showed that Plain CNN-56 struggled, while ResNet-56 remained much more stable and accurate.

There are some limitations in this project. First, all models were trained for only 20 epochs because of limited time and Colab resources. Second, we trained all models for 20 epochs using Adam optimizer, without extensive hyperparameter tuning, while the original ResNet paper used a different complex training setup.

For future work, we could test deeper models such as ResNet-110 and Plain CNN-110 to see if the gap becomes even clearer. Another direction would be to use CIFAR-100 or another harder dataset to test whether the same behavior appears with more classes.

## VIII. CONCLUSION

In this project, we compared Plain CNN and ResNet models for image classification on CIFAR-10. We first compared Plain CNN-18 with ResNet-18, then extended the experiment by adding Plain CNN-56 and ResNet-56. The purpose was to study whether residual connections help more when the network becomes deeper.

The results showed that ResNet-18 slightly outperformed Plain CNN-18, but the difference was not very large. However, when the depth increased to 56 layers, the difference became much clearer. Plain CNN-56 performed poorly compared to ResNet-56, showing that deeper plain networks can become difficult to train. ResNet-56 was able to handle the increased depth much better because of its shortcut connections. Overall, the project supports the main idea of ResNet: residual connections make deeper networks easier to optimize and more effective for image classification.

## REFERENCES


[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[2] S. Shin, S. D. Ho, and J. Kim, "Implementation and Comparison of CNN Models on CIFAR-10 Dataset," in *Proceedings of the International Conference on Industrial Engineering and Operations Management*, 2024.

[3] Y. Zheng, H. Huang, and J. Chen, "Comparative analysis of various models for image classification on Cifar-100 dataset," *Journal of Physics: Conference Series*, vol. 2711, 2024.

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, 2012.

[5] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

[6] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.